

Cybersecurity Lab Report

Port Scanning with RustScan

Target: Metasploitable2 (192.168.255.128) • Attacker: Kali Linux • Date: June 1, 2026

1. Introduction

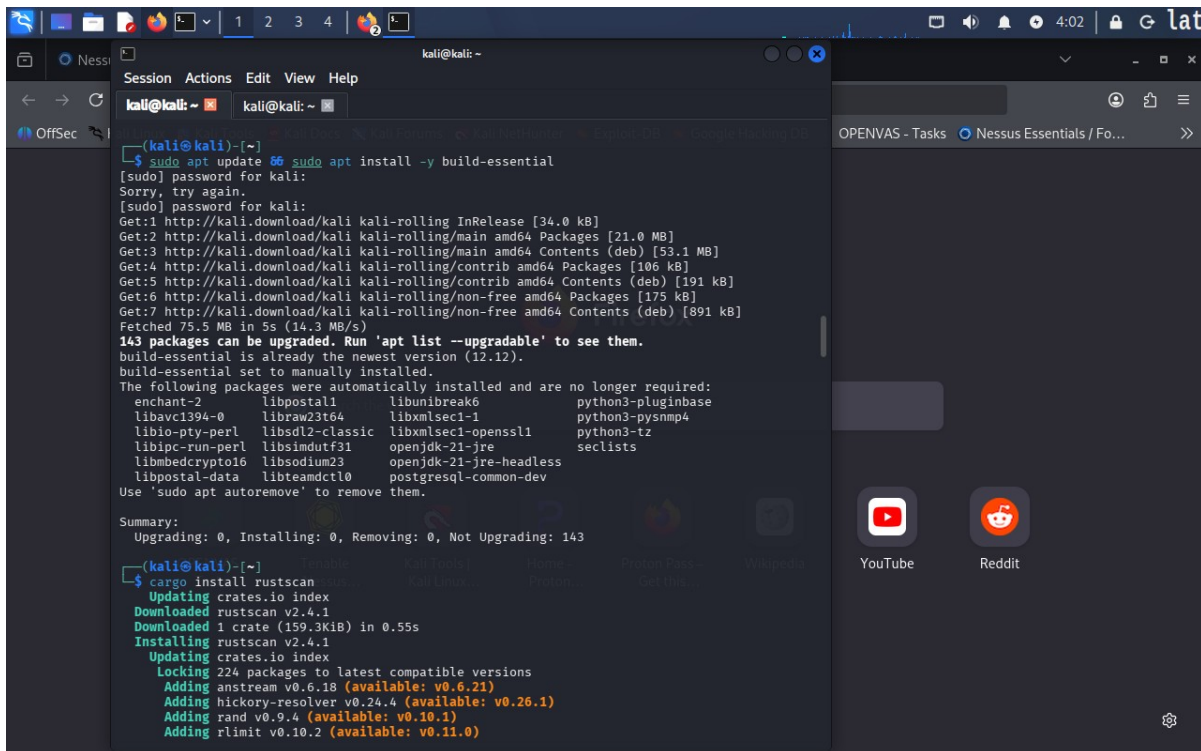
This report documents the use of RustScan, a modern high-speed port scanner written in Rust, as an alternative to Nmap for the initial reconnaissance phase of the lab. RustScan works by performing an ultra-fast TCP connect scan across all specified ports and then automatically passing the discovered open ports to Nmap for service detection and scripting. This two-phase approach combines RustScan's speed advantage with Nmap's depth of analysis.

The lab covers: installing the Rust toolchain and building RustScan from source via Cargo; running RustScan with varying batch sizes, ulimit values, and port ranges; interpreting the combined RustScan + Nmap output; and saving results to all Nmap output formats for later HTML conversion.

2. Installation

2.1 — Installing build-essential and Cargo (RustScan dependency)

Before installing RustScan, the Rust build toolchain was set up. First, ``sudo apt update && sudo apt install -y build-essential`` was run to ensure compilation tools were present. The terminal confirmed build-essential was already at the latest version (12.12) and that 143 packages were available for upgrade. Immediately after, ``cargo install rustscan`` downloaded and began installing RustScan v2.4.1 from the crates.io registry, downloading 1 crate (159.3 KiB) in 0.55 seconds and locking 224 dependency packages.



```
(kali@kali)~$ sudo apt update && sudo apt install -y build-essential
[sudo] password for kali:
Sorry, try again.
[sudo] password for kali:
Get:1 http://kali.download/kali kali-rolling InRelease [34.0 kB]
Get:2 http://kali.download/kali kali-rolling/main amd64 Packages [21.0 MB]
Get:3 http://kali.download/kali kali-rolling/main amd64 Contents (deb) [53.1 MB]
Get:4 http://kali.download/kali kali-rolling/contrib amd64 Packages [106 kB]
Get:5 http://kali.download/kali kali-rolling/contrib amd64 Contents (deb) [191 kB]
Get:6 http://kali.download/kali kali-rolling/non-free amd64 Packages [175 kB]
Get:7 http://kali.download/kali kali-rolling/non-free amd64 Contents (deb) [891 kB]
Fetched 75.5 MB in 5s (14.3 MB/s)
143 packages can be upgraded. Run 'apt list --upgradable' to see them.
build-essential is already the newest version (12.12).
build-essential set to manually installed.
The following packages were automatically installed and are no longer required:
  enchant-2      libpostall      libunibreak6    python3-pluginbase
  libavc1394-0  libraw23t64     libxmlsec1-1     python3-pysnmp4
  libio-pty-perl libssl2-classic libxmlsec1-openssl python3-tz
  libipc-run-perl libsimdutf31     openjdk-21-jre   seclists
  libmbcrypto16 libsodium23     openjdk-21-jre-headless
  libpostal-data libteamdctl0     postgresql-common-dev
Use 'sudo apt autoremove' to remove them.

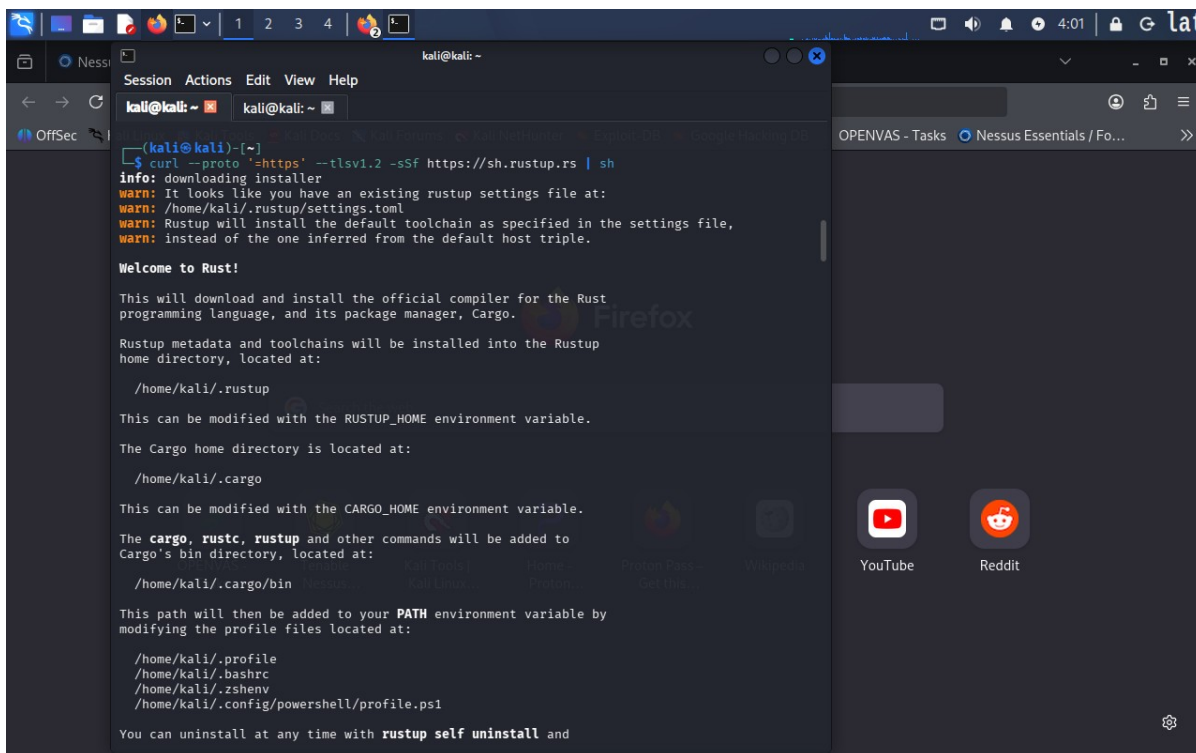
Summary:
Upgrading: 0, Installing: 0, Removing: 0, Not Upgrading: 143

(kali@kali)~$ cargo install rustscan
Updating crates.io index
Downloaded rustscan v2.4.1 (159.3 KiB in 0.55s)
Installing rustscan v2.4.1
Updating crates.io index
Locking 224 packages to latest compatible versions
Adding anstream v0.6.18 (available: v0.6.21)
Adding hickory-resolver v0.24.4 (available: v0.26.1)
Adding rand v0.9.4 (available: v0.10.1)
Adding rlimit v0.10.2 (available: v0.11.0)
```

Figure 1 — Kali terminal: ``sudo apt update && sudo apt install -y build-essential`` confirms build-essential 12.12 already installed, 143 packages upgradeable. Below, ``cargo install rustscan`` begins: Updating crates.io index, Downloaded rustscan v2.4.1 (159.3 KiB in 0.55s), Installing rustscan v2.4.1, Locking 224 packages, Adding anstream v0.6.18, hickory-resolver v0.24.4, rand v0.9.4, rlimit v0.10.2, toml v0.8.23 (newer versions noted).

2.2 — Installing the Rust Toolchain via rustup

In parallel, the official Rust toolchain installer was fetched with ``curl --proto '=https' --tlsv1.2 -sSf https://sh.rustup.rs | sh``. The installer displayed the "Welcome to Rust!" message, explained that Rustup metadata and toolchains would be installed to `/home/kali/.rustup`, that Cargo binaries (cargo, rustc, rustup) would be placed in `/home/kali/.cargo/bin`, and that this path would be added to the PATH variable via `/home/kali/.profile`, `.bashrc`, `.zshenv`, and the PowerShell profile.



```
(kali@kali)~$ curl --proto '=https' --tlsv1.2 -sSf https://sh.rustup.rs | sh
info: downloading installer
warn: It looks like you have an existing rustup settings file at:
warn: /home/kali/.rustup/settings.toml
warn: Rustup will install the default toolchain as specified in the settings file,
warn: instead of the one inferred from the default host triple.

Welcome to Rust!

This will download and install the official compiler for the Rust
programming language, and its package manager, Cargo.

Rustup metadata and toolchains will be installed into the Rustup
home directory, located at:

    /home/kali/.rustup

This can be modified with the RUSTUP_HOME environment variable.

The Cargo home directory is located at:

    /home/kali/.cargo

This can be modified with the CARGO_HOME environment variable.

The cargo, rustc, rustup and other commands will be added to
Cargo's bin directory, located at:

    /home/kali/.cargo/bin

This path will then be added to your PATH environment variable by
modifying the profile files located at:

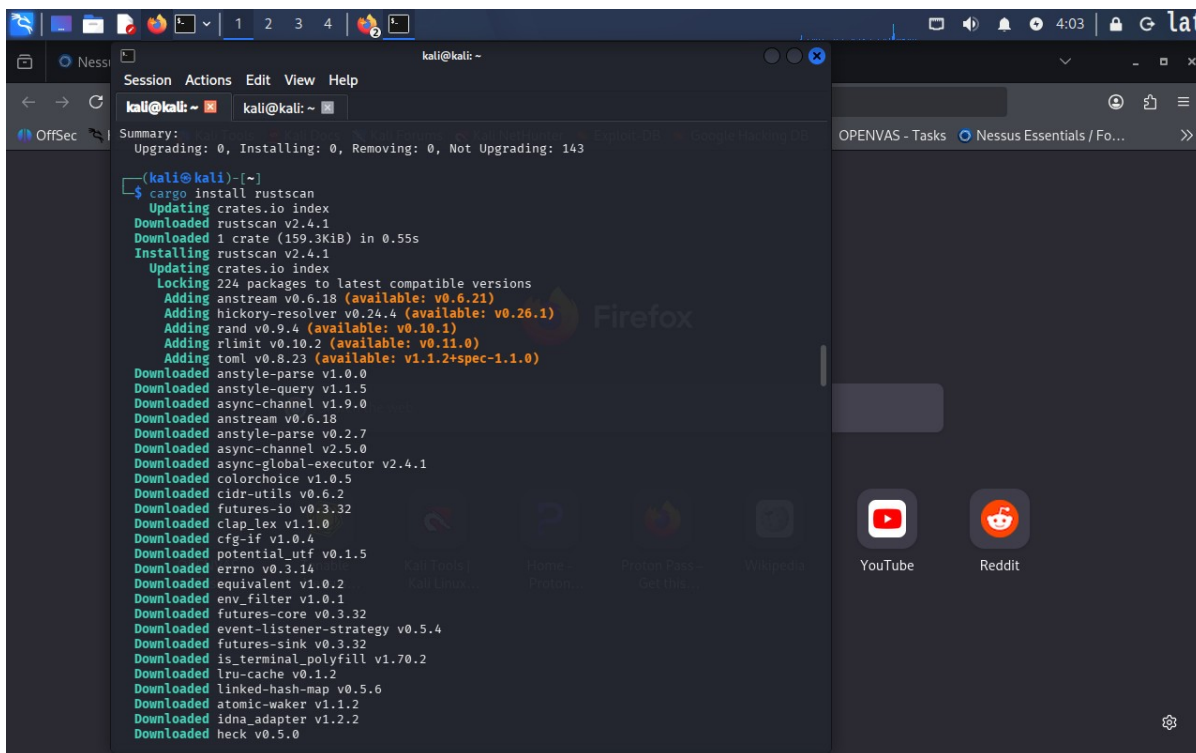
    /home/kali/.profile
    /home/kali/.bashrc
    /home/kali/.zshenv
    /home/kali/.config/powershell/profile.ps1

You can uninstall at any time with rustup self uninstall and
```

Figure 2 — Kali terminal: ``curl --proto '=https' --tlsv1.2 -sSf https://sh.rustup.rs | sh`` installer output. WARN: existing rustup settings.toml detected. "Welcome to Rust!" splash. Install paths listed: Rustup home /home/kali/.rustup, Cargo home /home/kali/.cargo, Cargo bin /home/kali/.cargo/bin. PATH modification files: .profile, .bashrc, .zshenv, powershell/profile.ps1. Instructions for uninstalling via ``rustup self uninstall`` shown.

2.3 — Cargo Dependency Download (RustScan Compilation)

The ``cargo install rustscan`` command compiled RustScan from source by downloading all required Rust crate dependencies. The terminal showed dozens of downloaded crates including anstyle-parse, anstyle-query, async-channel, anstream, colorschoice, cidr-utils, futures-io, clap_lex, cfg-if, potential_utf, errno, equivalent, env_filter, futures-core, event-listener-strategy, futures-sink, is_terminal_polyfill, lru-cache, linked-hash-map, atomic-waker, idna_adapter, and heck.



```
kali@kali: ~  
$ cargo install rustscan  
Summary:  
Upgrading: 0, Installing: 0, Removing: 0, Not Upgrading: 143  
  
(kali@kali)~$  
$ cargo install rustscan  
Updating crates.io index  
Downloaded rustscan v2.4.1  
Downloaded 1 crate (159.3KiB) in 0.55s  
Installing rustscan v2.4.1  
Updating crates.io index  
Locking 224 packages to latest compatible versions  
Adding anstream v0.6.18 (available: v0.6.21)  
Adding hickory-resolver v0.24.4 (available: v0.26.1)  
Adding rand v0.9.4 (available: v0.10.1)  
Adding rlimit v0.10.2 (available: v0.11.0)  
Adding toml v0.8.23 (available: v1.1.2+spec-1.1.0)  
Downloaded anstyle-parse v1.0.0  
Downloaded anstyle-query v1.1.5  
Downloaded async-channel v1.9.0  
Downloaded anstream v0.6.18  
Downloaded anstyle-parse v0.2.7  
Downloaded async-channel v2.5.0  
Downloaded async-global-executor v2.4.1  
Downloaded colorchoice v1.0.5  
Downloaded cidr-utils v0.6.2  
Downloaded futures-io v0.3.32  
Downloaded clap_lex v1.1.0  
Downloaded cfg-if v1.0.4  
Downloaded potential_utf v0.1.5  
Downloaded errno v0.3.14  
Downloaded equivalent v1.0.2  
Downloaded env_filter v1.0.1  
Downloaded futures-core v0.3.32  
Downloaded event-listener-strategy v0.5.4  
Downloaded futures-sink v0.3.32  
Downloaded is_terminal_polyfill v1.70.2  
Downloaded lru-cache v0.1.2  
Downloaded linked-hash-map v0.5.6  
Downloaded atomic-waker v1.1.2  
Downloaded idna_adapter v1.2.2  
Downloaded heck v0.5.0
```

Figure 3 — Kali terminal: continuation of ``cargo install rustscan`` showing the full dependency download list. Summary: Upgrading 0, Installing 0, Removing 0, Not Upgrading 143. Then ``cargo install rustscan`` starts: Updating crates.io index, Downloaded rustscan v2.4.1, 1 crate 159.3 KiB, Installing rustscan v2.4.1, Locking 224 packages. Dozens of crates downloaded including anstyle-parse v1.0.0, anstyle-query v1.1.5, async-channel v1.9.0, anstream v0.6.18, colorschoice v1.0.5, cidr-utils v0.6.2, futures-io v0.3.32, clap_lex v1.1.0, cfg-if v1.0.4, potential_utf v0.1.5, errno v0.3.14, equivalent v1.0.2, env_filter v1.0.1, futures-core v0.3.32, event-listener-strategy v0.5.4, futures-sink v0.3.32, is_terminal_polyfill v1.70.2, lru-cache v0.1.2, linked-hash-map v0.5.6, atomic-waker v1.1.2, idna_adapter v1.2.2, heck v0.5.0.

3. RustScan Scans

3.1 — First RustScan: Default Settings (`rustscan -a 192.168.255.128`)

The first RustScan was run with default settings: ``rustscan -a 192.168.255.128``. RustScan displayed its ASCII art logo and tagline "The Modern Day Port Scanner." It warned that the config file was missing, the file limit was lower than the default batch size, and that the system ulimit was very small. Despite these warnings, it successfully identified all 11 open ports and handed them off to Nmap for a full SYN stealth scan, producing the complete port table.

```
kali@kali: ~  
Session Actions Edit View Help  
kali@kali: ~ kali@kali: ~ kali@kali: ~  
kali@kali: ~  
$ rustscan -a 192.168.255.128  
RUSTSCAN/0.0.15  
The Modern Day Port Scanner.  
: http://discord.skerritt.blog  
: https://github.com/RustScan/RustScan :  
With RustScan, I scan ports so fast, even my firewall gets whiplash 🚀  
[~] The config file is expected to be at "/home/kali/.rustscan.toml"  
[!] File limit is lower than default batch size. Consider upping with --ulimit. May cause harm to sensitive servers  
[!] Your file limit is very small, which negatively impacts RustScan's speed. Use the Docker image, or up the Ulimit with '--ulimit 5000'.  
Open 192.168.255.128:139  
Open 192.168.255.128:445  
Open 192.168.255.128:2222  
Open 192.168.255.128:3307  
Open 192.168.255.128:3306  
Open 192.168.255.128:8080  
Open 192.168.255.128:8081  
Open 192.168.255.128:8084  
Open 192.168.255.128:8085  
Open 192.168.255.128:8082  
Open 192.168.255.128:8083  
[~] Starting Script(s)  
[~] Starting Nmap 7.99 ( https://nmap.org ) at 2026-06-01 04:09 -0400  
Initiating ARP Ping Scan at 04:09  
Scanning 192.168.255.128 [1 port]  
Completed ARP Ping Scan at 04:09, 0.05s elapsed (1 total hosts)  
Initiating Parallel DNS resolution of 1 host. at 04:09  
Completed Parallel DNS resolution of 1 host. at 04:09, 0.50s elapsed  
DNS resolution of 1 IPs took 0.50s. Mode: Async [#: 1, OK: 0, NX: 1, DR: 0, SF: 0, TR: 1, CN: 0]  
Initiating SYN Stealth Scan at 04:09  
Scanning 192.168.255.128 [11 ports]  
Discovered open port 3306/tcp on 192.168.255.128
```

Figure 4 — Kali terminal: `rustscan -a 192.168.255.128` — RustScan ASCII art logo displayed. "The Modern Day Port Scanner." Links: discord.skerritt.blog and github.com/RustScan/RustScan. Tagline: "With RustScan, I scan ports so fast, even my firewall gets whiplash." Three warnings: [~] config missing, [!] file limit lower than batch size, [!] ulimit very small. Open ports discovered (highlighted cyan): 139, 445, 2222, 3307, 3306, 8080, 8081, 8084, 8085, 8082, 8083. [~] Starting Scripts, [~] Starting Nmap 7.99 at 2026-06-01 04:09, ARP Ping Scan, Parallel DNS, SYN Stealth Scan on 11 ports, first open port 3306 discovered.

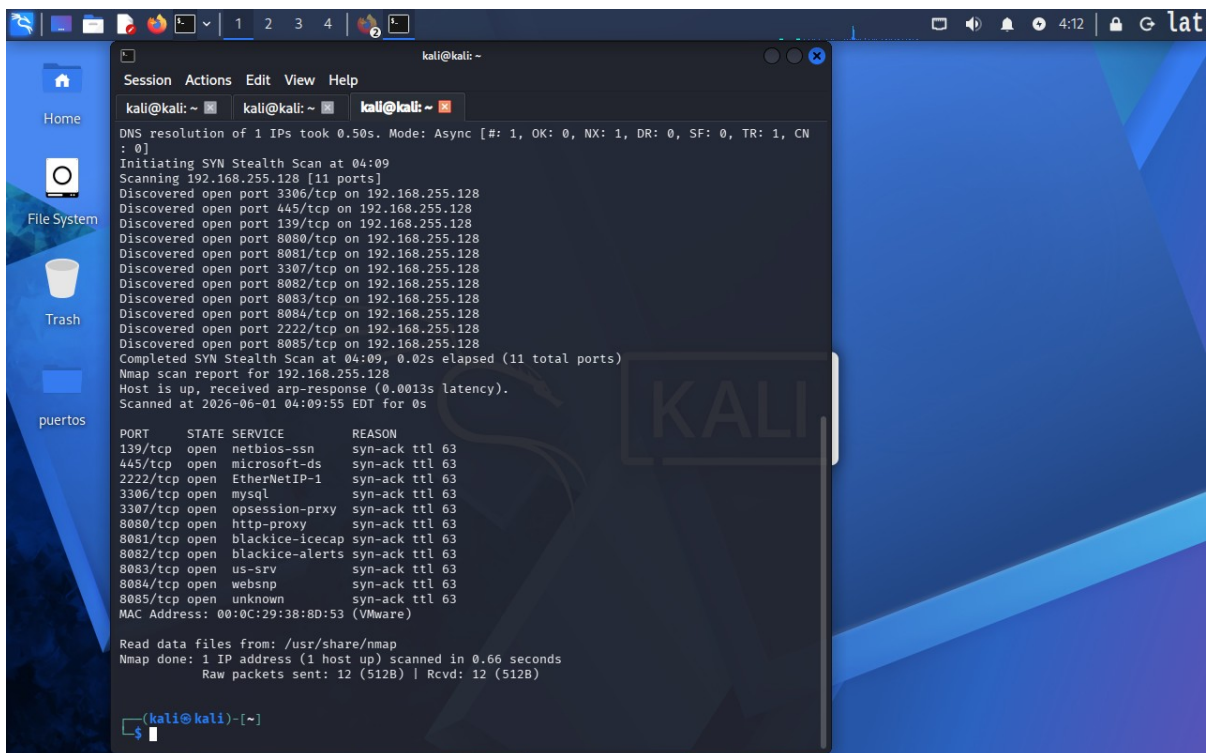
A screenshot of a Kali Linux desktop environment. The desktop background is a blue and black geometric pattern. On the left side, there is a sidebar with icons for Home, File System, Trash, and a folder named 'puertos'. The top of the window shows a taskbar with various application icons and a system tray on the right displaying the time as 4:12 and a 'lat' indicator. The main window is a terminal titled 'kali@kali: ~'. It displays the output of an Nmap scan performed by RustScan. The output includes DNS resolution details, scan initiation time (04:09), target IP (192.168.255.128), and a list of 11 discovered open ports. Below the port list, it shows the scan completion time (04:09), elapsed time (0.02s), and a full Nmap scan report for the same IP, which took 0.66 seconds. The report lists 11 open ports with their respective services and reasons. The MAC address is also displayed as 00:0C:29:38:8D:53 (VMware). The terminal prompt at the bottom is '(kali@kali)-[~]'.

Figure 5 — Kali terminal: Nmap output from the RustScan handoff — Nmap scan report for 192.168.255.128. Host is up (0.0013s latency). Scanned at 2026-06-01 04:09:55 EDT. Full port table: 139/tcp netbios-ssn syn-ack ttl 63, 445/tcp microsoft-ds, 2222/tcp EtherNet/IP-1, 3306/tcp mysql, 3307/tcp opsession-prxy, 8080/tcp http-proxy, 8081/tcp blackice-icecap, 8082/tcp blackice-alerts, 8083/tcp us-srv, 8084/tcp websnp, 8085/tcp unknown. MAC Address: 00:0C:29:38:8D:53 (VMware). Nmap done: 1 IP scanned in 0.66 seconds. Raw packets sent: 12 (512B), Rcvd: 12 (512B).

3.2 — Second Scan: Increased Ulimit (--ulimit 5000 -b 2500)

To fix the ulimit warning and improve scan speed, the second scan used `rustscan -a 192.168.255.128 --ulimit 5000 -b 2500`. RustScan confirmed "Automatically increasing ulimit value to 5000." and batch size to 2500, eliminating the previous warnings. The tagline changed to "Real hackers hack time." The same 11 ports were discovered and Nmap was again invoked.

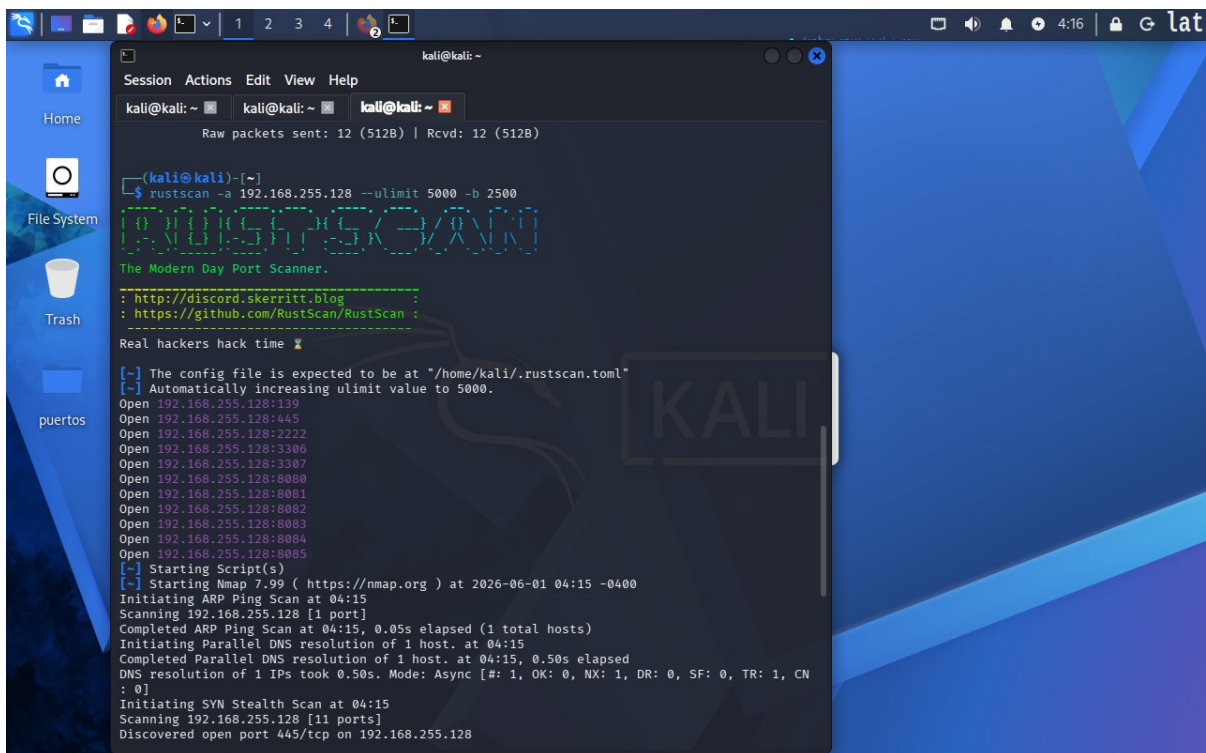


Figure 6 — Kali terminal: ``rustscan -a 192.168.255.128 --ulimit 5000 -b 2500`` — RustScan logo. Tagline: "Real hackers hack time." [~] Config file expected at `/home/kali/.rustscan.toml`; [~] Automatically increasing ulimit value to 5000. All 11 open ports discovered: 139, 445, 2222, 3306, 3307, 8080, 8081, 8082, 8083, 8084, 8085. [~] Starting Scripts, [~] Starting Nmap 7.99 at 04:15, ARP Ping Scan, Parallel DNS, SYN Stealth Scan on 11 ports, first open port 445 discovered.

3.3 — Third Scan: Port Range -r 1-1000

A targeted scan limiting the range to ports 1–1000 was run with ``rustscan -a 192.168.255.128 -r 1-1000``. This restricted the scan to the first 1000 ports. RustScan only discovered ports 445 and 139 in this range (both SMB-related), then passed them to Nmap. The tagline this run was "Scanning ports like it's my full-time job. Wait, it is."

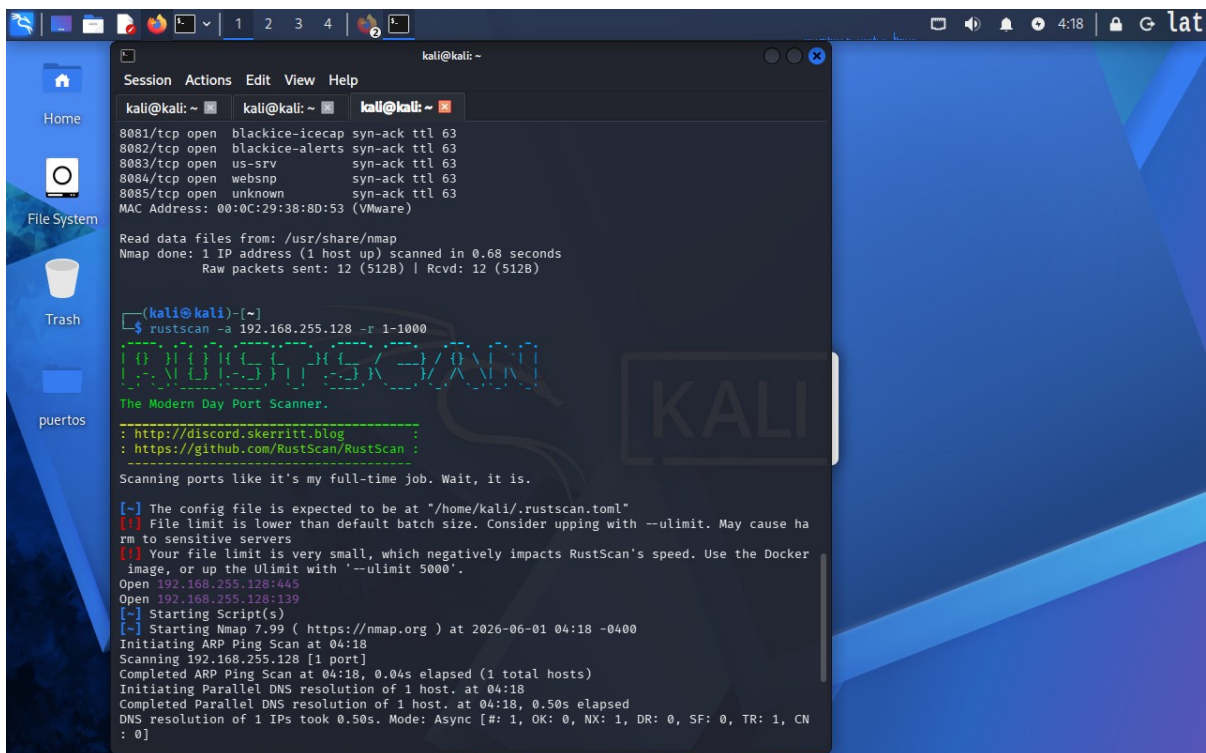


Figure 7 — Kali terminal: `rustscan -a 192.168.255.128 -r 1-1000` — RustScan logo. Tagline: "Scanning ports like it's my full-time job. Wait, it is." File limit and ulimit warnings present. Open ports in range 1–1000: 445 and 139 only (both SMB). [~] Starting Scripts, Nmap 7.99 at 04:18, ARP Ping Scan, 1 host, SYN Stealth Scan on 1 port range. Nmap done: 1 IP scanned in 0.68 seconds. Below: `rustscan -a 192.168.255.128 -r 1-5000` begins, Nmap invoked at 04:19, scanning 5 ports.

3.4 — Fourth Scan: Port Range -r 1-5000

Extending the range to `rustscan -a 192.168.255.128 -r 1-5000` captured additional ports beyond 1000. Tagline: "0day was here." Open ports found in this range: 139, 445, 2222, 3306, 3307.

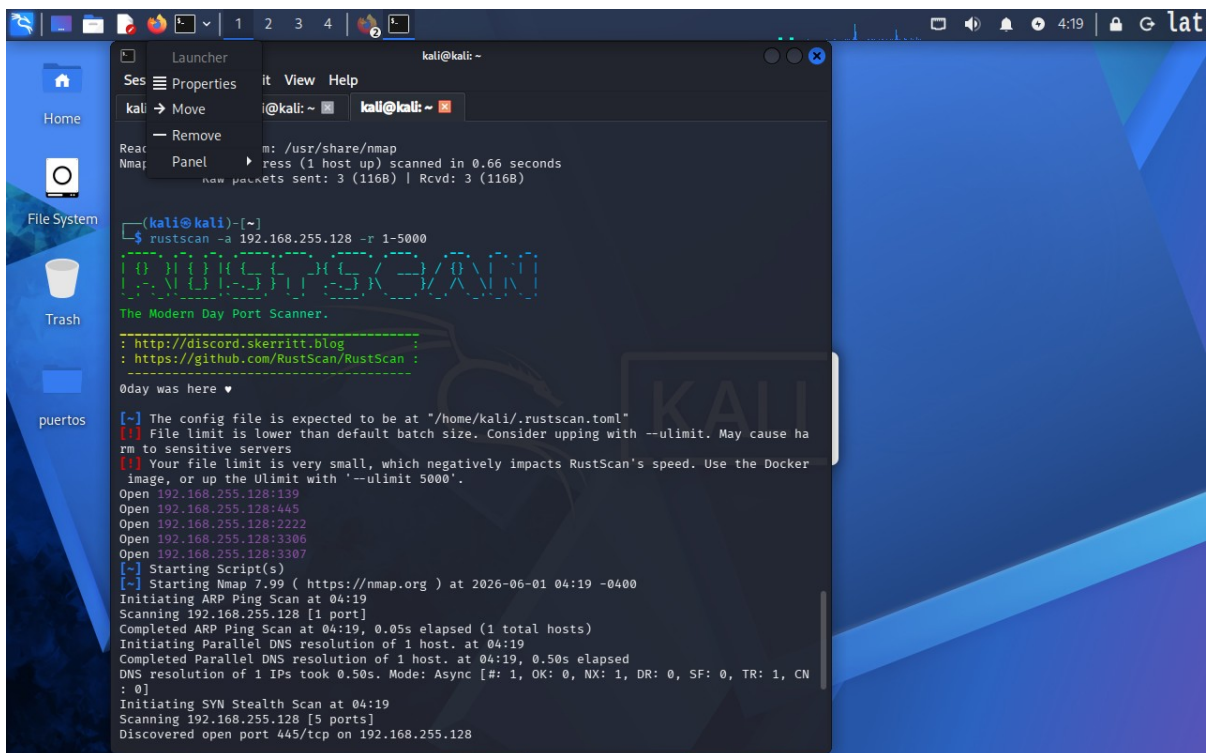


Figure 8 — Kali terminal: `rustscan -a 192.168.255.128 -r 1-5000` — RustScan logo. Tagline: "Oday was here ♥". File/ulimit warnings. Open ports 139, 445, 2222, 3306, 3307 discovered. Nmap invoked at 04:19 scanning 5 ports, first discovered port 445. A context menu (Launcher, Properties, Move, Remove, Panel) is briefly visible in the taskbar area, indicating an accidental right-click on the desktop panel.

3.5 — Fifth Scan: Port Range -r 1-9000

Expanding further with `rustscan -a 192.168.255.128 -r 1-9000`, RustScan discovered all 11 target ports within the 1–9000 range. Tagline: "Nmap? More like slowmap."

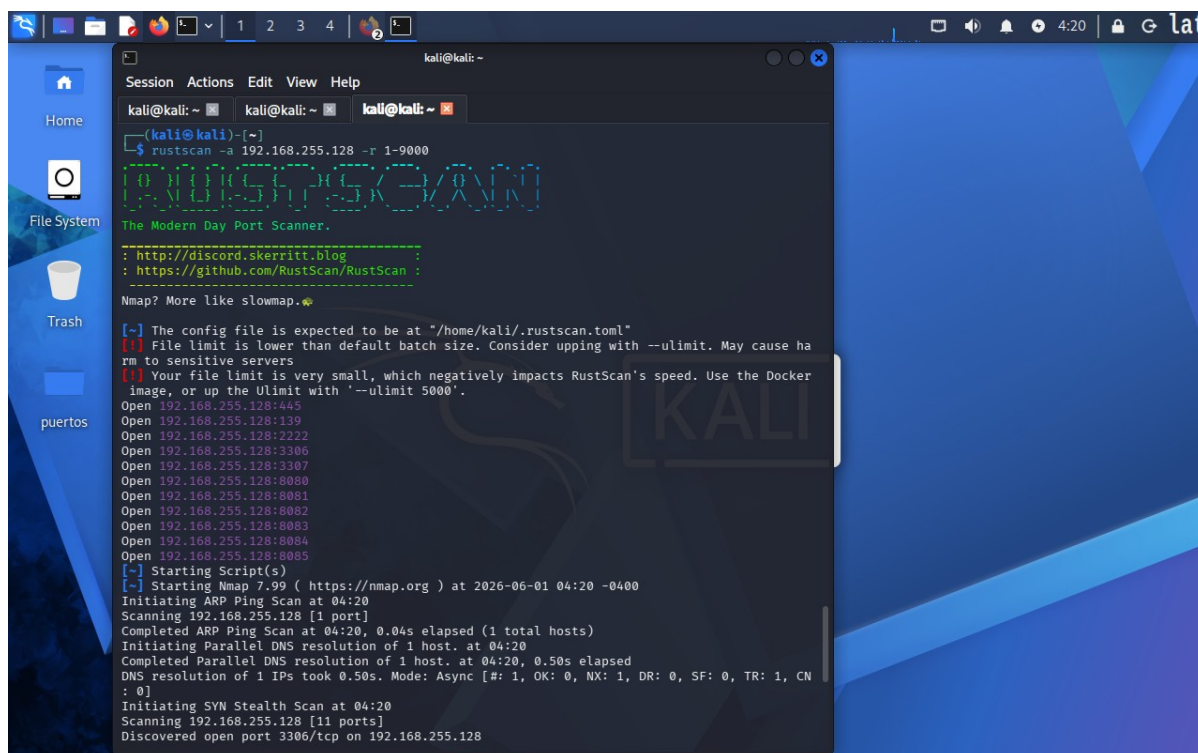


Figure 9 — Kali terminal: `rustscan -a 192.168.255.128 -r 1-9000` — RustScan logo. Tagline: "Nmap? More like slowmap." All 11 ports discovered: 445, 139, 2222, 3306, 3307, 8080, 8081, 8082, 8083, 8084, 8085. Nmap invoked at 04:20, ARP Ping Scan (0.04s), Parallel DNS (0.50s), SYN Stealth Scan on 11 ports, first open port 3306.

3.6 — Sixth Scan: Combined -sV -sC Flags (Service + Script Detection)

Running `rustscan -a 192.168.255.128 -- -sV -sC` passed Nmap the service version and default script flags after the double dash separator. RustScan tagline: "RustScan: allowing you to send UDP packets into the void 1200x faster than NMAP." This triggered a deeper scan including NSE scripts for all discovered ports.

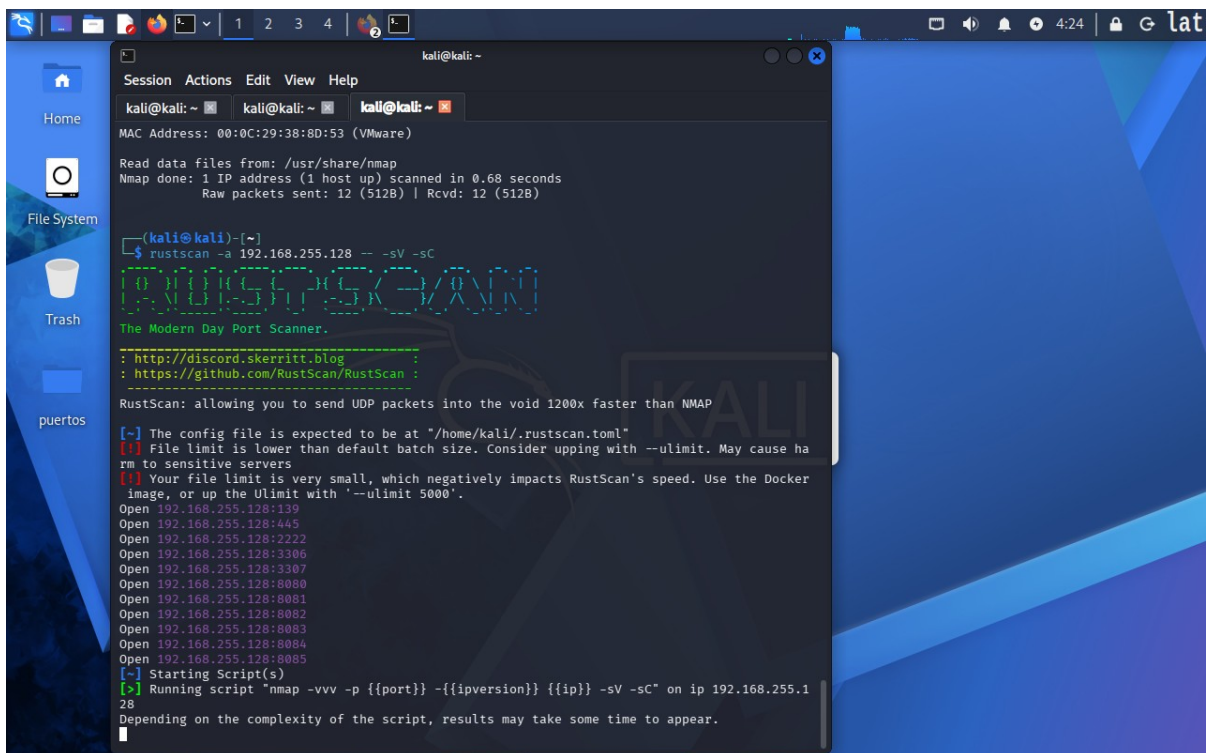


Figure 10 — Kali terminal: ``rustscan -a 192.168.255.128 -- -sV -sC`` — RustScan logo. Tagline: "RustScan: allowing you to send UDP packets into the void 1200x faster than NMAP." All 11 ports open: 139, 445, 2222, 3306, 3307, 8080, 8081, 8082, 8083, 8084, 8085. [~] Starting Script(s), [>] Running script "nmap -vvv -p {{port}} -{{ipversion}} {{ip}} -sV -sC" on 192.168.255.128. "Depending on the complexity of the script, results may take some time to appear."

3.7 — Saving Output: -oA salidarust Flag + HTML Report

The most comprehensive scan saved all Nmap output formats using ``rustscan -a 192.168.255.128 -- -sV -sC -oA salidarust``. After the scan completed, ``xsltproc salidarust.xml -o salidarust.html`` converted the XML output to an HTML report and ``firefox salidarust.html`` opened it in the browser. Thunar showed the rustscan subdirectory with `salidarust.gnmap`, `salidarust.nmap`, and `salidarust.xml` created.

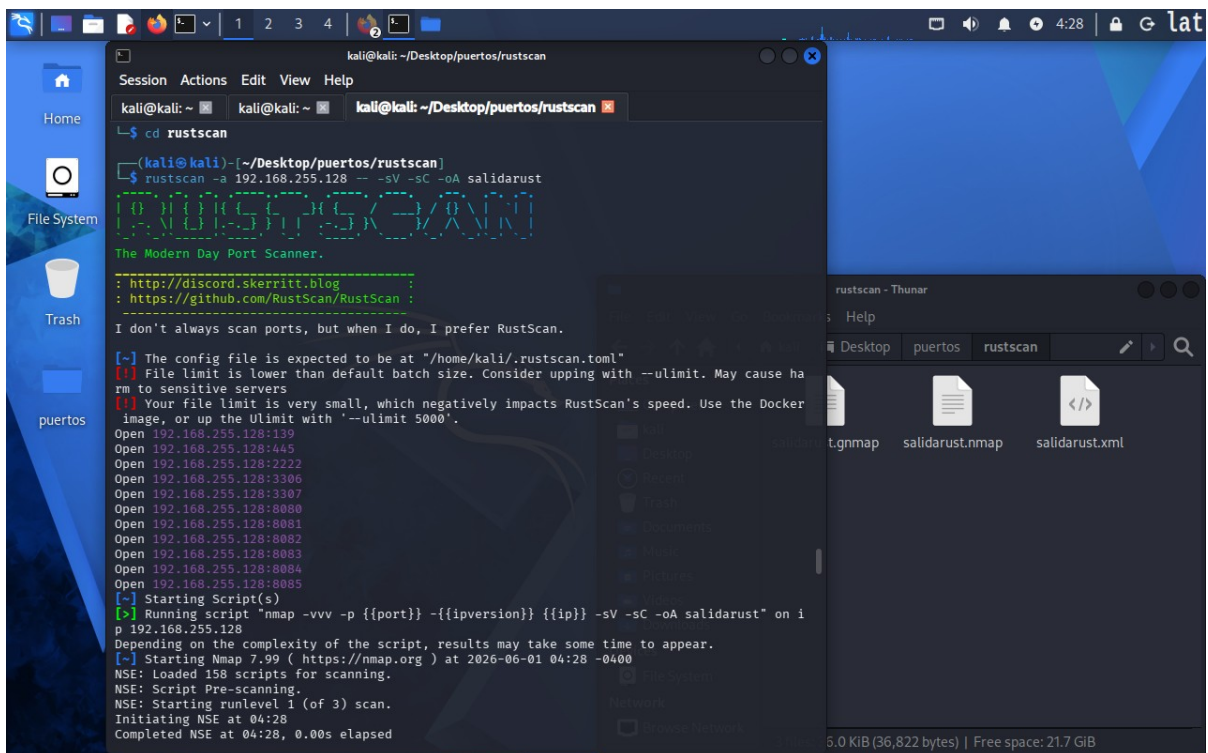


Figure 11 — Kali terminal (`~/Desktop/puertos/rustscan`): ``rustscan -a 192.168.255.128 -- -sV -sC -oA salidarust``. RustScan logo. Tagline: "I don't always scan ports, but when I do, I prefer RustScan." All 11 open ports listed. [~] Starting Scripts(s), [>] Running "nmap -vvv -p {{port}} -{{ipversion}} {{ip}} -sV -sC -oA salidarust" on 192.168.255.128. NSE loaded 158 scripts. Thunar file manager shows the rustscan folder with: (partial .gnmap visible), salidarust.nmap, salidarust.xml (3 files, 36,822 bytes, 21.7 GiB free).

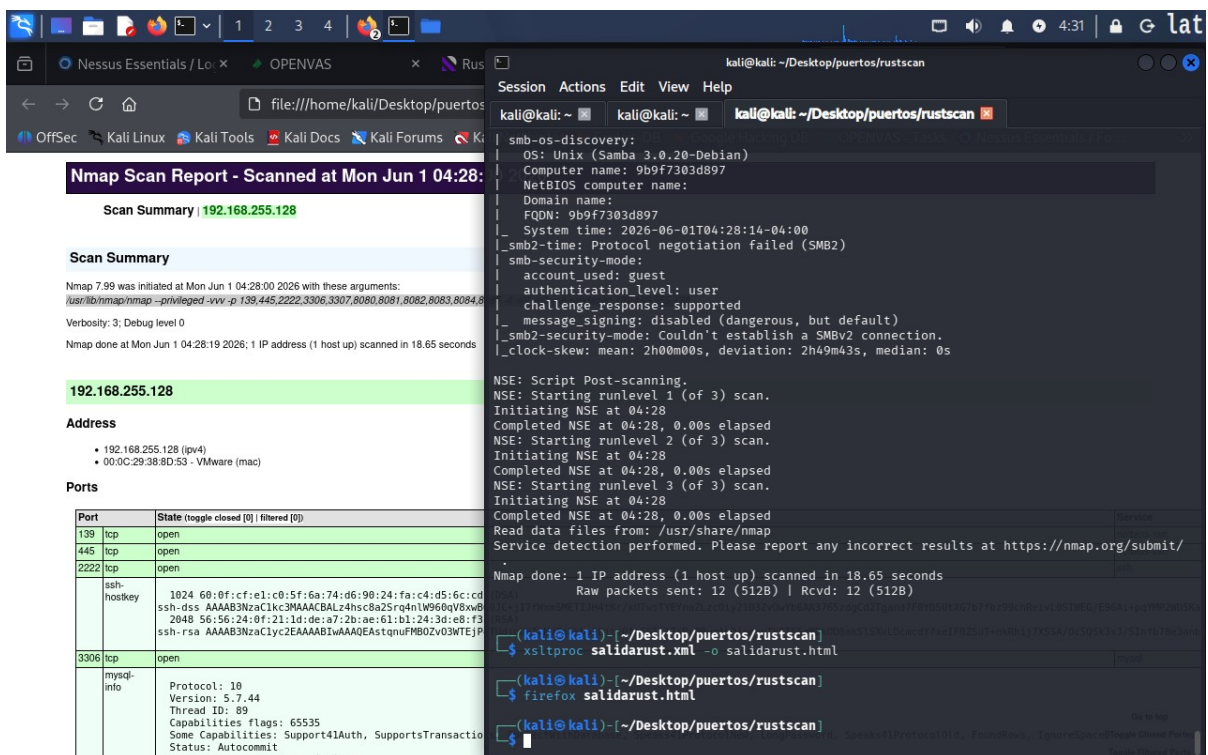


Figure 12 — Kali terminal + Firefox (split view): Left side — terminal showing Nmap -sV -sC scan results with SMB OS discovery (Samba 3.0.20-Debian, computer name 9b9f7303d897, FQDN 9b9f7303d897, system time 2026-06-01T04:28:14), smb2-time protocol negotiation failed, smb-security-mode (account_used: guest, authentication_level: user, challenge_response: supported, message_signing: disabled), clock-skew: 2h00m00s. NSE post-scanning phases. Nmap done: 1 IP, 18.65 seconds, 12 raw packets. Then: `xsltproc salidarust.xml -o salidarust.html` and `firefox salidarust.html` commands. Right side — Firefox displaying the Nmap Scan Report HTML: "Scanned at Mon Jun 1 04:28:00 2026", Scan Summary for 192.168.255.128, address 192.168.255.128 (ipv4) and 00:0C:29:38:8D:53 VMware (mac), Ports table showing 139/tcp, 445/tcp, 2222/tcp open with ssh-hostkey details, 3306/tcp MySQL 5.7.44 with capabilities info.